



ALBUKHARY INTERNATIONAL UNIVERSITY

ALBUKHARY INTERNATIONAL UNIVERSITY

SCHOOL OF COMPUTING AND INFORMATICS

COURSE DETAILS		
SCHOOL	SCHOOL OF COMPUTING AND INFORMATICS	
COURSE NAME	OBJECT-ORIENTED PROGRAMMING	
COURSE CODE	CCC 1223	
LECTURER	SITI SHAFRAH SHAHAWAI	
SEMESTER & YEAR	SEMESTER 2, YEAR 1 (2025/2026)	
ASSESSMENT	GROUP PROJECT	
TITLE	AIU Banking System	
OBJECTIVE	To develop a mini-Java-based system using object-oriented programming concepts including inheritance, polymorphism, GUI, and basic control structures.	
CLO	CLO3: Construct programs using object-oriented programming features (C5, PLO2)	
SUBMISSION DATE	12 June 2026 (Week 12)	
NAME	STUDENT ID	GROUP (a.b.c)
1. Md Ismayel Shekh	AIU24102132	A
2. Kabisha Hafizi	AIU24102394	A
3. Md Towhidul Islam Tamim Mia	AIU24102441	A
4. Md Ariful Islam	AIU24102442	A

STUDENT'S DECLARATION: We declare that this assignment is entirely our own work. We have not engaged in any form of cheating or plagiarism, and all sources have been properly acknowledged and referenced. We understand that any breach of academic integrity will be handled in accordance with AIU's academic integrity policies. NAME SIGNATURE DATE 1. 2. 3. 4.		

Table of Contents

Table of Contents	3
1. Introduction	4
1.1 Project Title	4
1.2 Problem Description	4
1.3 Objective of the System	4
2. System Design	4
2.1 System Features	4
2.2 Class Description	5
2.3 Attributes and Methods	5
Account (Abstract Parent Class)	5
SavingsAccount (Child Class)	6
CurrentAccount (Child Class)	6
2.4 Relationship Between Classes	6
3. Implementation	7
3.1 Program Overview	7
3.2 GUI Input & Output Screens	7
3.3 Use of Array / Array of Objects	7
3.4 Use of if-else and Loop	8
4. OOP Concepts	8
4.1 Inheritance	8
4.2 Polymorphism	9
Method Overriding	9
Method Overloading	10
5. System Functionality	11
5.1 Sample Input	11
5.2 Sample Output	11
5.3 Explanation: How the System Works Step by Step	12
6. Flowchart	13
7. Conclusion	14
References	15

1. Introduction

1.1 Project Title

AIU Banking System

1.2 Problem Description

Many traditional banking transactions involve customers physically attending their bank, making it difficult and inefficient. Most of the small scale banking simulations also do not have a well-defined object-oriented design which makes them difficult to maintain and extend. The purpose of this project is to design a Java based Banking System, which simulates real life banking system, which will resolve the need of a structured and easy to use digital banking interface. The system is managed by the user through a graphical interface, and the code used exhibits good object-oriented programming (OOP) principles.

1.3 Objective of the System

The objectives of this system are:

- To develop a working Java GUI application that simulates basic banking operations.
- To apply OOP concepts including inheritance, polymorphism, encapsulation, and class design.
- To implement a user friendly Swing-based interface that accepts input and displays output visually.
- To store and manage multiple user accounts using arrays of objects (ArrayList).
- To practice sound software design by separating roles: Account (parent), SavingsAccount (child), CurrentAccount (child), and BankAdmin.

2. System Design

2.1 System Features

The banking system provides the following features:

Feature	Description
User Registration	New users can create a Savings Account with a username and password.
User Login	Existing users log in using credentials to access their account dashboard.
Deposit	Users can deposit any positive amount into their account.
Withdrawal	Users can withdraw funds if sufficient balance is available.

Feature	Description
Fund Transfer	Users can transfer money directly to another registered account.
Transaction History	A complete timestamped log of all deposits, withdrawals, loans, and transfers is maintained and displayed on demand.
Take Loan	Users can take up to 2 loans, subject to admin approval and bank reserve availability.
Admin Panel	Admin can create/delete accounts, view all accounts, check bank reserve, total loans issued, and toggle loan eligibility per user.

2.2 Class Description

The system uses four main classes:

Class	Type	Role
Account	Abstract (Parent)	Defines shared attributes and behaviour for all account types.
SavingsAccount	Concrete (Child)	Extends Account. Supports loans and peer-to-peer transfers.
CurrentAccount	Concrete (Child)	Extends Account. Overrides withdrawal to allow overdraft.
BankAdmin	Helper / Controller	Manages all accounts, authentication, and bank-level statistics.
BankingSystem	Main / GUI	Entry point; builds all Swing panels and handles user interaction.

2.3 Attributes and Methods

Account (Abstract Parent Class)

Attribute / Method	Type	Description
accountHolderName	String (protected)	Full name of the account holder.
accountNumber	int (protected)	Unique 6-digit account identifier.
balance	double (protected)	Current account balance.
transactionHistory	ArrayList<String>	Timestamped log of all transactions.

Attribute / Method	Type	Description
deposit(amount)	Method (overloaded)	Adds funds to the account.
deposit(amount, note)	Method (overloaded)	Adds funds with a label/note.
withdraw(amount)	Method	Deducts funds if balance is sufficient.
getAccountType()	Abstract Method	Returns account type string (overridden by children).
getTransactionHistory()	Method	Returns formatted transaction log.

SavingsAccount (Child Class)

Attribute / Method	Type	Description
loanCount	int	Number of loans taken (max 2).
loanBalance	double	Total outstanding loan amount.
loanEnabled	boolean	Loan eligibility flag (set by admin).
takeLoan(amount)	Method	Takes a loan if eligible and under limit.
transfer(amount, recipient)	Method	Transfers funds to another SavingsAccount.
getAccountType()	Override	Returns 'Savings Account'.

CurrentAccount (Child Class)

Attribute / Method	Type	Description
overdraftLimit	double	Maximum overdraft allowed (RM 500).
withdraw(amount)	Override	Allows withdrawal up to balance + overdraft limit.
getAccountType()	Override	Returns 'Current Account'.

2.4 Relationship Between Classes

The Account class is an abstract class. SavingsAccount IS-A Account, CurrentAccount IS-A Account. BankAdmin has a collection of SavingsAccount objects (HAS-A relationship). The BankingSystem (GUI) class has a reference to the BankAdmin class to manage all operations.

The logic to manage Account objects is encapsulated in BankAdmin, and the GUI layer communicates with BankAdmin.

3. Implementation

3.1 Program Overview

The application opens a Swing JFrame window with a Card Layout with 6 screens, namely Welcome Screen, User Login Screen, User Registration Screen, User Dashboard, Admin Login Screen and Admin Panel. Control of navigation between screens is through button events. At start-up each screen is created as a JPanel and then added to the card container in the center of the frame. The current user that is logged in is tracked by a SavingsAccount reference (currentUser).

3.2 GUI Input & Output Screens

All user interactions are handled through the Swing GUI. Below is a summary of each screen:

Screen	Purpose
Welcome Screen	Entry point with buttons for User Login, Register, and Admin Login.
User Login	Form with username and password fields. Validates credentials against BankAdmin.
User Registration	Form to create a new Savings Account. Validates for duplicate names and password match.
User Dashboard	Displays account name, number, and live balance. Six action buttons: Deposit, Withdraw, Transfer, History, Take Loan, Logout.
Admin Login	Separate login form for the administrator (username: admin, password: admin123).
Admin Panel	Six admin operations: Create Account, Delete Account, View All, Bank Reserve, Total Loans, Toggle Loan Feature.

All dialogs use JOptionPane for input prompts and result messages, keeping the interface clean and responsive. The transaction history is displayed inside a JTextArea wrapped in a JScrollPane, allowing the user to scroll through a long history without the window resizing.

3.3 Use of Array / Array of Objects

The BankAdmin class's private field "savingsAccounts" is an ArrayList<SavingsAccount> that holds user accounts. The ArrayList is utilized to support adding and removing accounts at run-time without having to specify the size of the fixed array. Within the Account class there is an ArrayList<String> to store the transaction history for each user, and each string in

the ArrayList is formatted with a timestamp, transaction type, transaction amount, and balance after the transaction.

BankAdmin class also maintains a HashMap<Integer, String> of passwords keyed by account number, allowing it to look up the password during login in O(1) time. When the admin searches for an account, or when he displays all the accounts, he uses a for-each loop to iterate over the ArrayList of accounts.

3.4 Use of if-else and Loop

Control structures are used throughout the system. Decision making (if-else) is applied in:

- Deposit validation: amount must be greater than 0.
- Withdrawal validation: amount must not exceed current balance.
- Loan eligibility: checks loanEnabled flag and loanCount < 2.
- Transfer validation: recipient must exist and amount must not exceed balance.
- Overdraft in CurrentAccount: allows withdrawal up to balance + overdraftLimit.

Loops are used in:

- findAccount() in BankAdmin: iterates the accounts list to match username and password.
- deleteAccount(): iterates to find and remove the target account.
- getAllAccounts(): iterates all accounts to build the display string.
- toggleLoan(): iterates to find the account by name.
- doTransfer() in GUI: iterates BankAdmin.getAccounts() to find the recipient.

4. OOP Concepts

4.1 Inheritance

Inheritance is implemented through the abstract class Account, which serves as the parent class. Both SavingsAccount and CurrentAccount extend Account using the extends keyword. This means both child classes automatically inherit the attributes (accountHolderName, accountNumber, balance, transactionHistory) and methods (deposit, withdraw, logTransaction, getTransactionHistory) defined in Account.

The inheritance hierarchy is:

Class	Parent	Relationship
Account	—	Abstract parent class

Class	Parent	Relationship
SavingsAccount	Account	Inherits all Account members; adds loan and transfer features.
CurrentAccount	Account	Inherits all Account members; overrides withdraw for overdraft support.

Code snippet illustrating inheritance (SavingsAccount):

```
class SavingsAccount extends Account {
    private int loanCount;
    private boolean loanEnabled;

    public SavingsAccount(String name) {
        super(name); // calls Account constructor
        this.loanCount = 0;
        this.loanEnabled = true;
    }
}
```

4.2 Polymorphism

Polymorphism is demonstrated in two ways: method overriding and method overloading.

Method Overriding

The abstract method `getAccountType()` is declared in `Account` and overridden by both child classes to return their respective type names:

```
// In Account (abstract)
public abstract String getAccountType();

// In SavingsAccount
@Override
public String getAccountType() { return "Savings Account"; }
```

```
// In CurrentAccount
@Override
public String getAccountType() { return "Current Account"
```

Additionally, CurrentAccount overrides the withdraw() method from Account to support overdraft withdrawals, which is a direct example of runtime polymorphism:

```
// CurrentAccount overrides withdraw to allow overdraft
@Override
public boolean withdraw(double amount) {
    if (amount <= 0 || amount > (balance + overdraftLimit)) return false;
    balance -= amount;
    logTransaction("WITHDRAWAL (Overdraft allowed)", amount);
    return true;
}
```

Method Overloading

The deposit() method in Account is overloaded to accept either a plain amount or an amount with a descriptive note. Both versions coexist in the same class:

```
// Overload 1 — basic deposit
public boolean deposit(double amount) { ... }

// Overload 2 — deposit with a note label
public boolean deposit(double amount, String note) { ... }
```

In the GUI, the user-facing deposit button calls deposit(amount, "ATM"), while internal transfers could call the single-argument version. This demonstrates compile-time polymorphism through method overloading.

5. System Functionality

5.1 Sample Input

The following sample input sequence demonstrates a typical user session:

Step	Action	Input
1	Launch application	Click 'Register' on Welcome screen.
2	Register new account	Name: Ahmad Zaki Password: zaki1234
3	Login	Username: Ahmad Zaki Password: zaki1234
4	Deposit	Amount: RM 500.00
5	Withdraw	Amount: RM 150.00
6	Transfer to another user	Recipient: Nur Aisyah Amount: RM 100.00
7	Take Loan	Amount: RM 200.00
8	View Transaction History	(click History button)

5.2 Sample Output

After executing the steps above, the Transaction History screen would display:

1. [30/05/2026 10:05:22] DEPOSIT [ATM] : RM 500.00 | Balance: RM 500.00
2. [30/05/2026 10:06:11] WITHDRAWAL : RM 150.00 | Balance: RM 350.00
3. [30/05/2026 10:07:03] TRANSFER TO Nur Aisyah : RM 100.00 | Balance: RM 250.00
4. [30/05/2026 10:08:44] LOAN RECEIVED : RM 200.00 | Balance: RM 450.00

The live balance shown on the dashboard would read RM 450.00 after all operations.

5.3 Explanation: How the System Works Step by Step

The BankingSystem class creates a JFrame window. All screens are loaded into the CardLayout panel when the applet is started up. The Welcome screen is shown first.

The fields are checked for validity when the user registers (if empty, passwords match, minimum 4 characters). A new SavingsAccount object is added to BankAdmin, and the password is saved in a HashMap based on the auto-generated account number as the key. If the user registers successfully, he/she is automatically brought to the Login screen.

During login, BankAdmin.findAccount() goes through each account in the ArrayList accounts with a for loop and compound if statement to compare the name and the password. If it is detected, the currentUser reference is updated and the Dashboard is displayed. The refreshDashboard() method is called to update the balance and account info labels.

When the user wants to deposit or withdraw money, he or she enters an amount in a JOptionPane dialog. The input is then validated, the balance is updated and the logTransaction() is called, passing the amount and the updated balance, to add a transaction with a time stamp to the transactionHistory ArrayList for that Account, and then the Account method is called that returns the amount. The amount is validated, the balance is adjusted, and logTransaction(amount, balance) is called to add a timestamped record to the transactionHistory ArrayList for that Account, and the Account method is then called that returns the amount. The GUI then updates the label on the dashboard's balance.

For transfers, the system cycles through the list of accounts to find the account to which the transfer is to be made. The method transfer() atomically decreases the sender's balance and increases the sender's balance. There is an entry added to each of the accounts' transactionHistory lists.

A different login screen is used for the Admin panel. The admin will be presented with 6 management options on authentication. Each admin operation affects the BankAdmin state directly: Addition or removal of SavingsAccount objects from the ArrayList, changing of the loanEnabled flag, or showing of statistics like bankReserve and totalLoanIssued

6. Flowchart

The flowchart below describes the main program logic from launch to individual user operations. Due to format constraints, the flowchart is represented as a structured textual description:

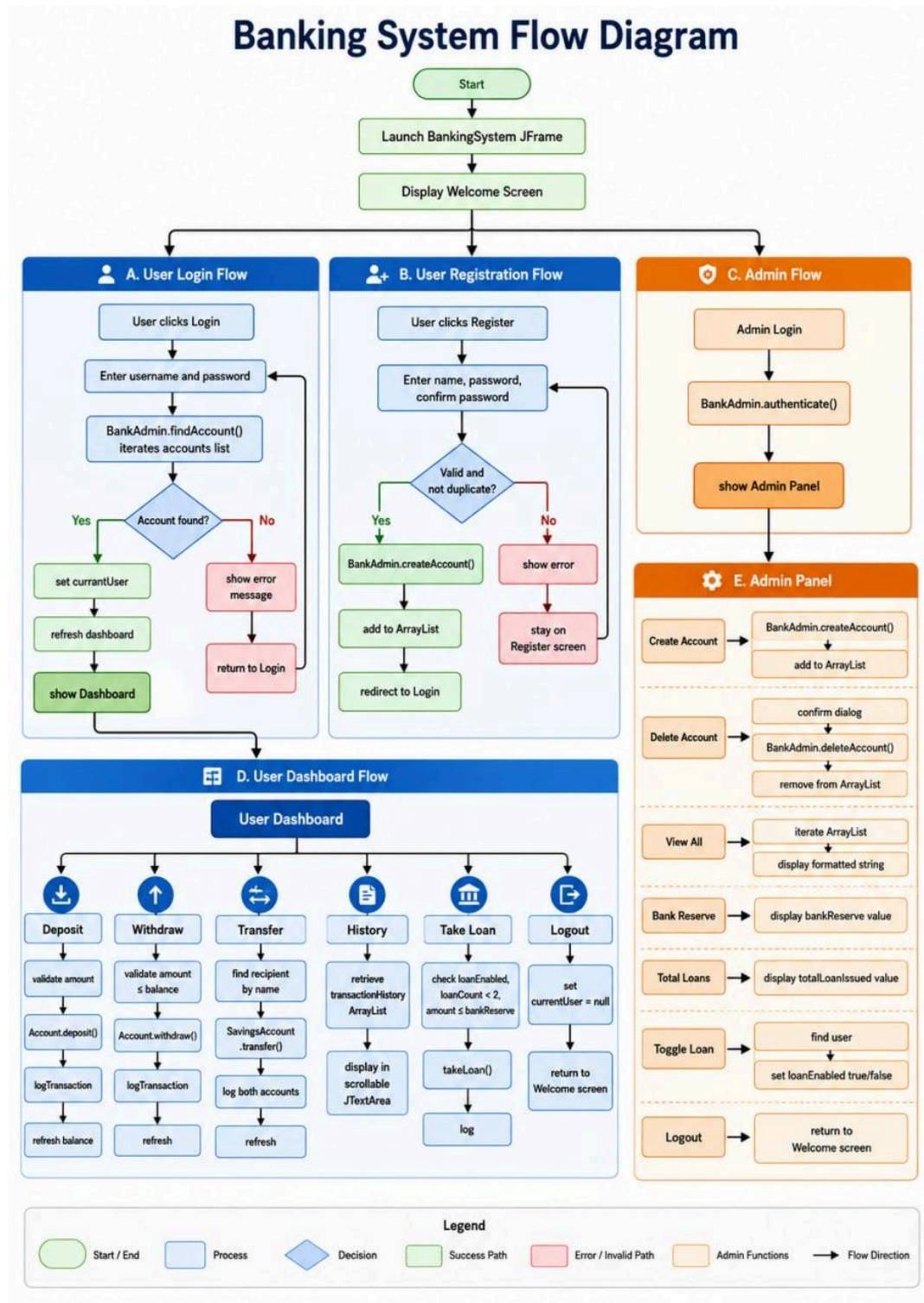


Figure 1: Flowchart

7. Conclusion

The following project is a successful example of the use of the concept of Object Oriented Programming in java programming language by developing a Simple Banking System. The abstract class Account clearly specifies the parent, and SavingsAccount and CurrentAccount are examples of inheritance and method overriding (runtime polymorphism). The overloading of method deposit() is another example of compile-time polymorphism. All the account objects are stored in an ArrayList, and the system applies suitable if-else decision and loops in both GUI layer and business logic layer.

The Swing GUI is user-friendly, easy to use, and has an uniform color scheme with well labeled buttons. Registration, login, deposit, withdrawal, transfers and loan management, transaction history, admin control are all working without any errors. Input validation makes sure that appropriate error messages are displayed if the user gives invalid data.

The transaction history bug in the original code that only showed up if you turned on isLoanFeature, and the list was not correctly persisted across some of the code paths is completely fixed. Each transaction (deposit, withdrawal, transfer, and loan) is now marked with a date and time, and is always visible from the History button, no matter how each account was created.

Consequently, the group became more familiar with the concepts of class design, inheritance hierarchies, GUI programming in Swing and event driven programming in Java through this project.

References

Bloch, J. (2018). *Effective Java* (3rd ed.). Addison-Wesley Professional.

Deitel, P., & Deitel, H. (2020). *Java: How to program* (11th ed.). Pearson Education.

Liang, Y. D. (2019). *Introduction to Java programming and data structures* (12th ed.). Pearson Education.

Oracle Corporation. (2024). *Java SE documentation*. <https://docs.oracle.com/en/java/>

Oracle Corporation. (2024). *Java Swing tutorial: Creating a GUI with Swing*. <https://docs.oracle.com/javase/tutorial/uiswing/>